

Algorithms





Minimum Spanning Tree

Prim's and Kruskal's Algorithm



Trees

Recall

Definition (Trees)

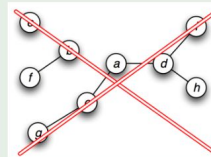
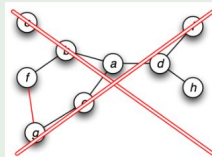
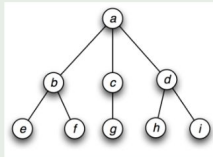
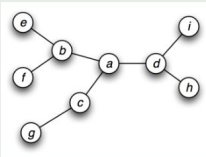
A undirected graph is a *tree* if it is *connected* and does not contain a *cycle* (implies that every n -node tree has exactly $n - 1$ edges).

Theorem

For undirected graph G , any two of the following implies the third.

- 1 G is connected.
- 2 G does not contain a cycle.
- 3 G has $n - 1$ edges.

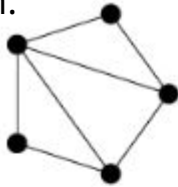
Example (Trees (or not))



Difference Between Tree and Spanning Tree

- A tree is a type of graph. A spanning tree is a subgraph of the graph that is a tree and hits every vertex.

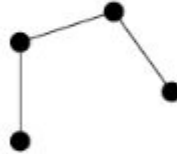
This graph:



has the spanning tree



whereas the subgraph



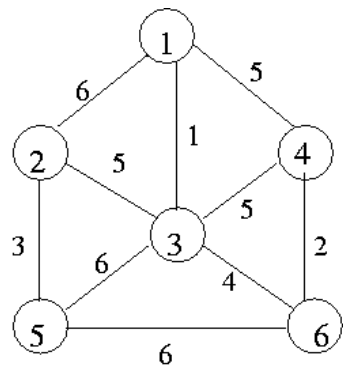
is not a spanning tree (it's a tree, but it's not spanning). The subgraph



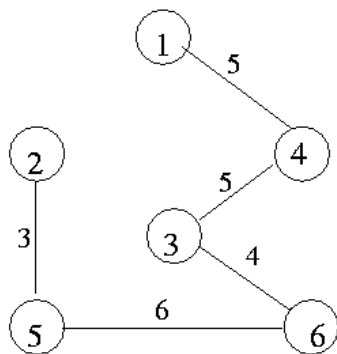
is also not a spanning tree (it's spanning, but it's not a tree).

Minimum Spanning Tree (MST)

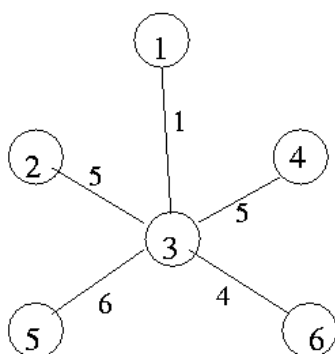
- Minimum spanning tree is the spanning tree where the cost is minimum among all the spanning trees.



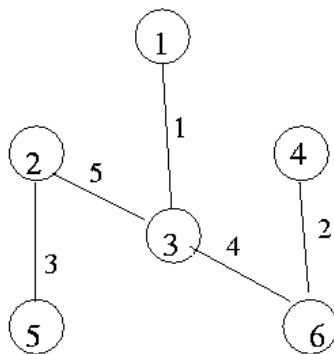
A connected graph.



A spanning tree with cost = 23



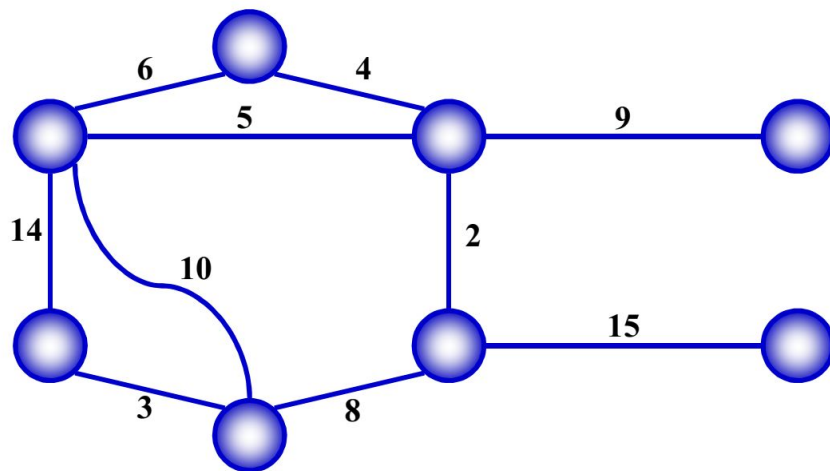
Another spanning tree with cost 21



MST, cost = 15

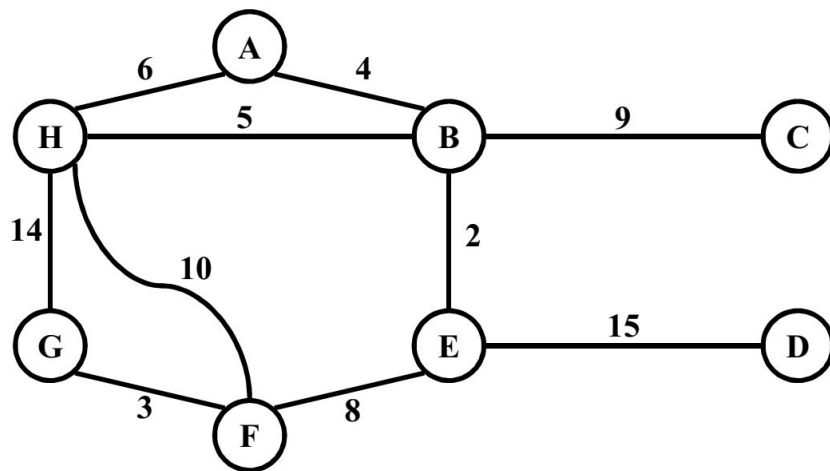
Minimum Spanning Tree

- Problem: given a connected, undirected, weighted graph, find a *spanning tree* using edges that **minimize** the total weight



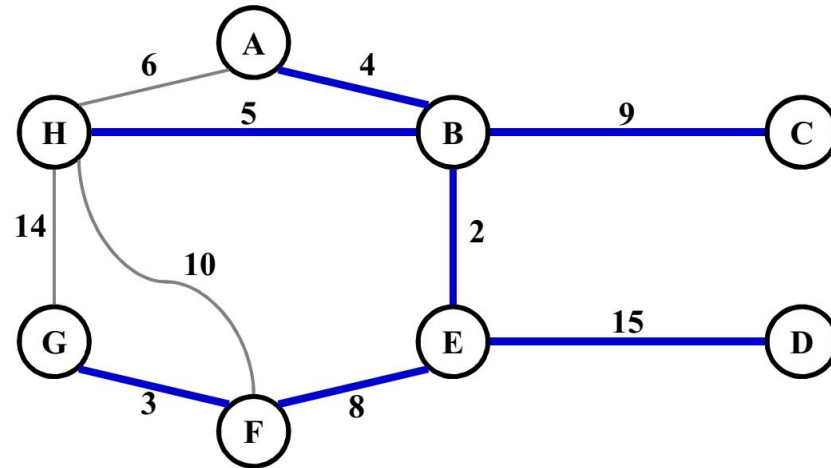
Minimum Spanning Tree

- Which edges form the minimum spanning tree (MST) of the graph as shown below?



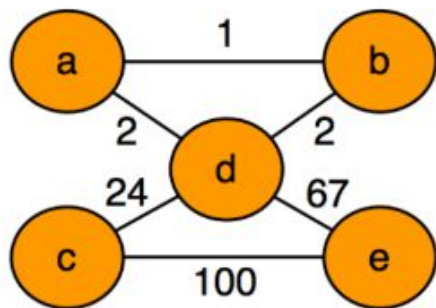
Minimum Spanning Tree

- Answer:

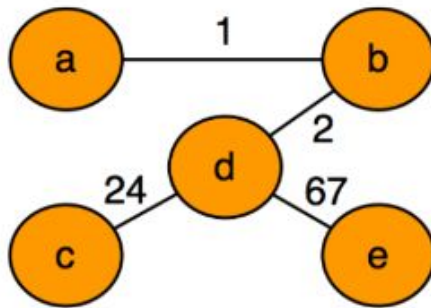


Uniqueness of MST

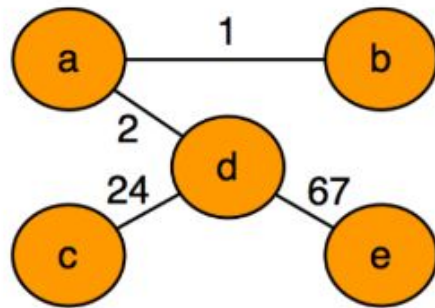
The minimum-cost spanning tree may not be unique!



Graph G



$$\sum w_T = 94$$



$$\sum w_T = 94$$

Key observation

However, if the weights are all distinct (i.e., $w(u_i, v_j) \neq w(u_k, v_l)$ unless $i = k$ and $j = l$), then it is indeed unique.

Computing an MST

- We **grow** the tree one edge at a time, starting with a graph $G' = (V, \emptyset)$.
- At each step, add a new **safe edge**, ensuring that it does not create a cycle (why?).
- If adding an edge guarantees that the tree after each step is a subset of some MST, then the final result will be an MST.

Key question

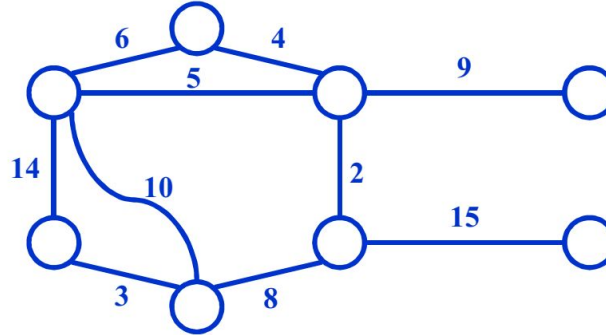
How do we pick the next **safe edge**?

Prim's Algorithm

```
MST-Prim( $G, w, r$ )
   $Q = V[G];$ 
  for each  $u \in Q$ 
     $key[u] = \infty;$ 
   $key[r] = 0;$ 
   $p[r] = \text{NULL};$ 
  while ( $Q$  not empty)
     $u = \text{ExtractMin}(Q);$ 
    for each  $v \in \text{Adj}[u]$ 
      if ( $v \in Q$  and  $w(u, v) < key[v]$ )
         $p[v] = u;$ 
         $key[v] = w(u, v);$ 
```

Prim's Algorithm

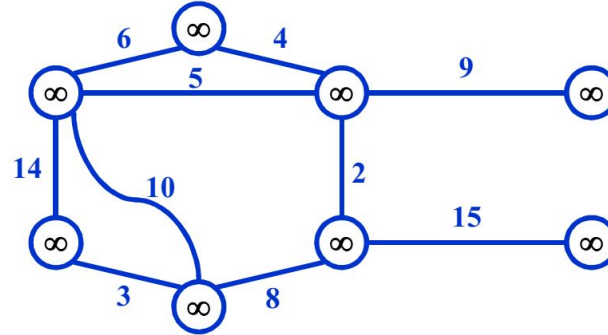
```
MST-Prim( $G, w, r$ )  
   $Q = V[G]$ ;  
  for each  $u \in Q$   
     $\text{key}[u] = \infty$ ;  
   $\text{key}[r] = 0$ ;  
   $p[r] = \text{NULL}$ ;  
  while ( $Q$  not empty)  
     $u = \text{ExtractMin}(Q)$ ;  
    for each  $v \in \text{Adj}[u]$   
      if ( $v \in Q$  and  $w(u,v) < \text{key}[v]$ )  
         $p[v] = u$ ;  
         $\text{key}[v] = w(u,v)$ ;
```



Run on example graph

Prim's Algorithm

```
MST-Prim(G, w, r)
  Q = V[G];
  for each u ∈ Q
    key[u] = ∞;
  key[r] = 0;
  p[r] = NULL;
  while (Q not empty)
    u = ExtractMin(Q);
    for each v ∈ Adj[u]
      if (v ∈ Q and w(u,v) < key[v])
        p[v] = u;
        key[v] = w(u,v);
```



Run on example graph

Prim's Algorithm

```
MST-Prim( $G, w, r$ )
```

```
   $Q = V[G];$ 
```

```
  for each  $u \in Q$ 
```

```
     $\text{key}[u] = \infty;$ 
```

```
   $\text{key}[r] = 0;$ 
```

```
   $p[r] = \text{NULL};$ 
```

```
  while ( $Q$  not empty)
```

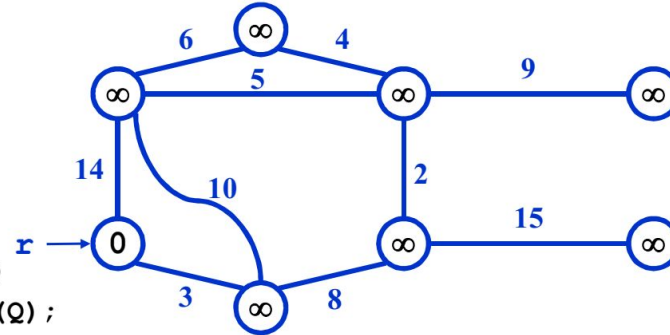
```
     $u = \text{ExtractMin}(Q);$ 
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u,v) < \text{key}[v]$ )
```

```
         $p[v] = u;$ 
```

```
         $\text{key}[v] = w(u,v);$ 
```



Prim's Algorithm

```
MST-Prim( $G, w, r$ )
```

```
   $Q = V[G];$ 
```

```
  for each  $u \in Q$ 
```

```
     $\text{key}[u] = \infty;$ 
```

```
   $\text{key}[r] = 0;$ 
```

```
   $p[r] = \text{NULL};$ 
```

```
  while ( $Q$  not empty)
```

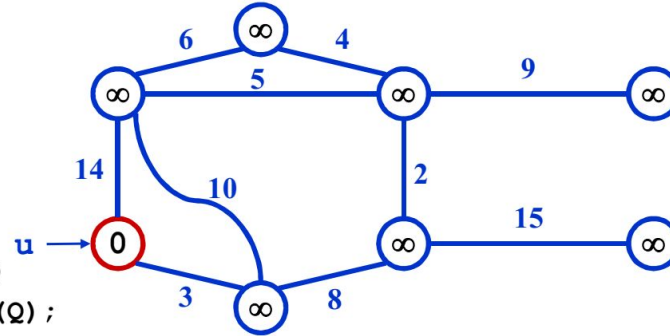
```
     $u = \text{ExtractMin}(Q);$ 
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u,v) < \text{key}[v]$ )
```

```
         $p[v] = u;$ 
```

```
         $\text{key}[v] = w(u,v);$ 
```



Red vertices have been removed from Q

Prim's Algorithm

```
MST-Prim( $G, w, r$ )
```

```
   $Q = V[G];$ 
```

```
  for each  $u \in Q$ 
```

```
     $\text{key}[u] = \infty;$ 
```

```
   $\text{key}[r] = 0;$ 
```

```
   $p[r] = \text{NULL};$ 
```

```
  while ( $Q$  not empty)
```

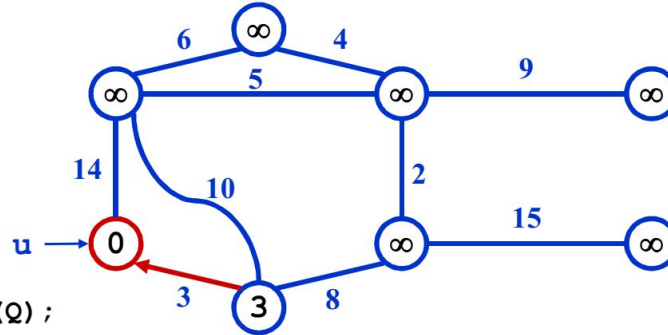
```
     $u = \text{ExtractMin}(Q);$ 
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u,v) < \text{key}[v]$ )
```

```
         $p[v] = u;$ 
```

```
         $\text{key}[v] = w(u,v);$ 
```



Prim's Algorithm

```
MST-Prim(G, w, r)
```

```
  Q = V[G];
```

```
  for each  $u \in Q$ 
```

```
     $\text{key}[u] = \infty$ ;
```

```
   $\text{key}[r] = 0$ ;
```

```
   $p[r] = \text{NULL}$ ;
```

```
  while (Q not empty)
```

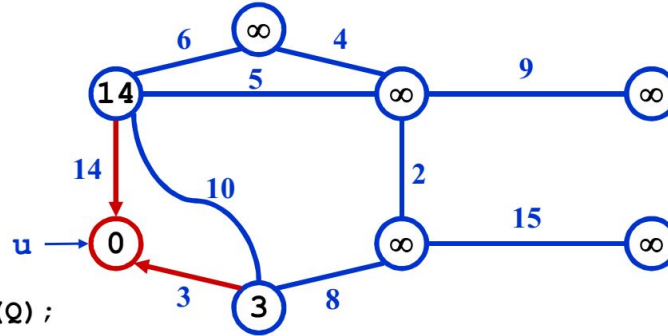
```
     $u = \text{ExtractMin}(Q)$ ;
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )
```

```
         $p[v] = u$ ;
```

```
         $\text{key}[v] = w(u, v)$ ;
```



Prim's Algorithm

```
MST-Prim( $G, w, r$ )
```

```
   $Q = V[G];$ 
```

```
  for each  $u \in Q$ 
```

```
     $\text{key}[u] = \infty;$ 
```

```
   $\text{key}[r] = 0;$ 
```

```
   $p[r] = \text{NULL};$ 
```

```
  while ( $Q$  not empty)
```

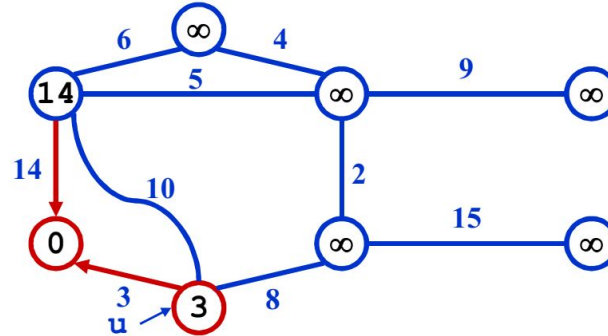
```
     $u = \text{ExtractMin}(Q);$ 
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )
```

```
         $p[v] = u;$ 
```

```
         $\text{key}[v] = w(u, v);$ 
```



Prim's Algorithm

```
MST-Prim(G, w, r)
```

```
  Q = V[G];
```

```
  for each  $u \in Q$ 
```

```
     $\text{key}[u] = \infty$ ;
```

```
   $\text{key}[r] = 0$ ;
```

```
   $p[r] = \text{NULL}$ ;
```

```
  while (Q not empty)
```

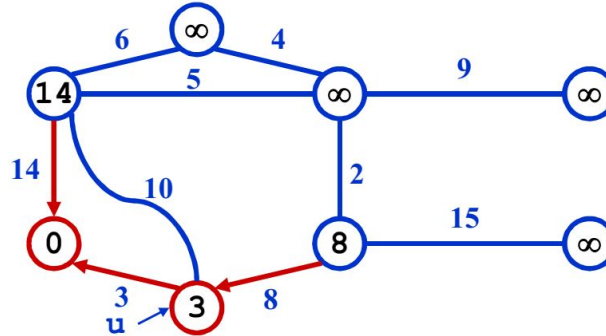
```
     $u = \text{ExtractMin}(Q)$ ;
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )
```

```
         $p[v] = u$ ;
```

```
         $\text{key}[v] = w(u, v)$ ;
```



Prim's Algorithm

```
MST-Prim(G, w, r)
```

```
  Q = V[G];
```

```
  for each u ∈ Q
```

```
    key[u] = ∞;
```

```
  key[r] = 0;
```

```
  p[r] = NULL;
```

```
  while (Q not empty)
```

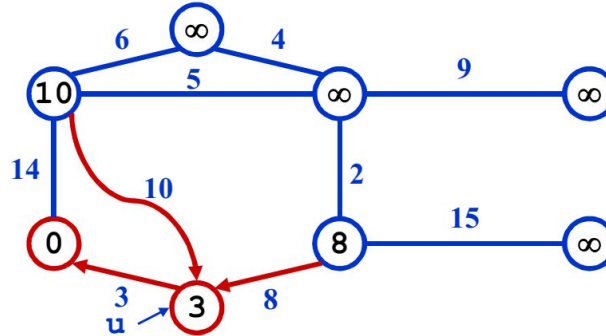
```
    u = ExtractMin(Q);
```

```
    for each v ∈ Adj[u]
```

```
      if (v ∈ Q and w(u,v) < key[v])
```

```
        p[v] = u;
```

```
        key[v] = w(u,v);
```



Prim's Algorithm

```
MST-Prim( $G, w, r$ )
```

```
   $Q = V[G];$ 
```

```
  for each  $u \in Q$ 
```

```
     $\text{key}[u] = \infty;$ 
```

```
   $\text{key}[r] = 0;$ 
```

```
   $p[r] = \text{NULL};$ 
```

```
  while ( $Q$  not empty)
```

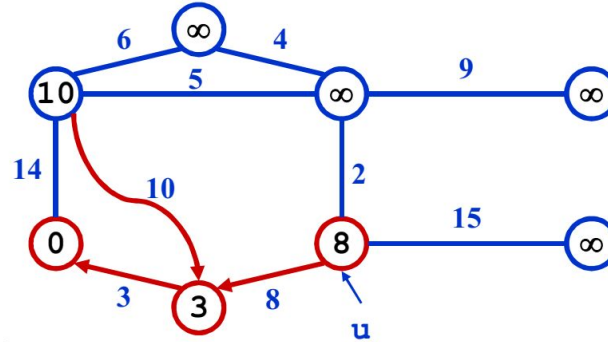
```
     $u = \text{ExtractMin}(Q);$ 
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )
```

```
         $p[v] = u;$ 
```

```
         $\text{key}[v] = w(u, v);$ 
```



Prim's Algorithm

```
MST-Prim( $G, w, r$ )
```

```
   $Q = V[G];$ 
```

```
  for each  $u \in Q$ 
```

```
     $\text{key}[u] = \infty;$ 
```

```
   $\text{key}[r] = 0;$ 
```

```
   $p[r] = \text{NULL};$ 
```

```
  while ( $Q$  not empty)
```

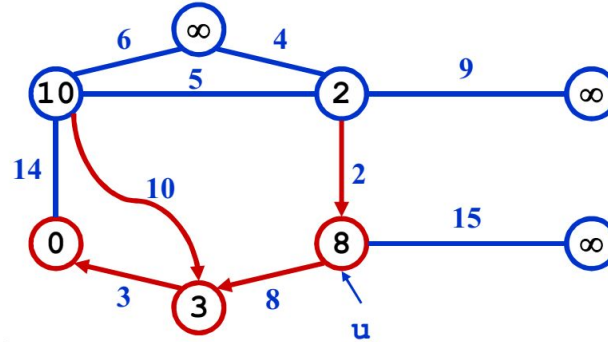
```
     $u = \text{ExtractMin}(Q);$ 
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )
```

```
         $p[v] = u;$ 
```

```
         $\text{key}[v] = w(u, v);$ 
```



Prim's Algorithm

```
MST-Prim(G, w, r)
```

```
  Q = V[G];
```

```
  for each u ∈ Q
```

```
    key[u] = ∞;
```

```
  key[r] = 0;
```

```
  p[r] = NULL;
```

```
  while (Q not empty)
```

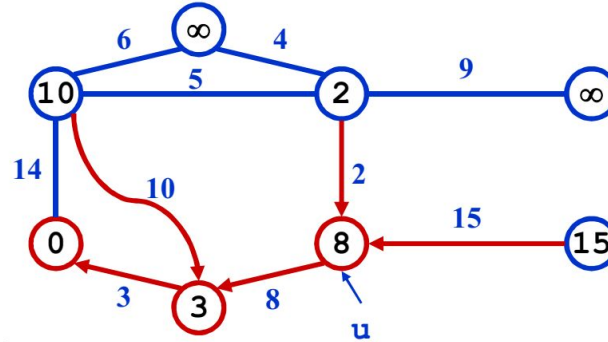
```
    u = ExtractMin(Q);
```

```
    for each v ∈ Adj[u]
```

```
      if (v ∈ Q and w(u,v) < key[v])
```

```
        p[v] = u;
```

```
        key[v] = w(u,v);
```



Prim's Algorithm

```
MST-Prim(G, w, r)
```

```
  Q = V[G];
```

```
  for each u ∈ Q
```

```
    key[u] = ∞;
```

```
  key[r] = 0;
```

```
  p[r] = NULL;
```

```
  while (Q not empty)
```

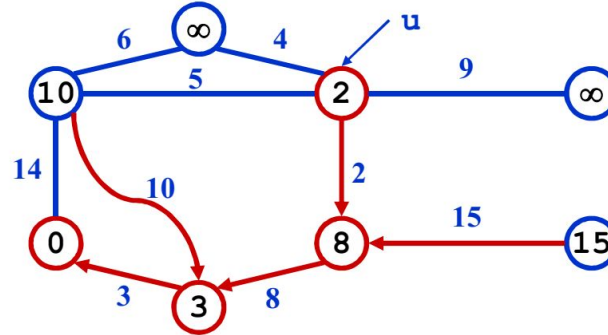
```
    u = ExtractMin(Q);
```

```
    for each v ∈ Adj[u]
```

```
      if (v ∈ Q and w(u,v) < key[v])
```

```
        p[v] = u;
```

```
        key[v] = w(u,v);
```



Prim's Algorithm

```
MST-Prim(G, w, r)
```

```
  Q = V[G];
```

```
  for each u ∈ Q
```

```
    key[u] = ∞;
```

```
  key[r] = 0;
```

```
  p[r] = NULL;
```

```
  while (Q not empty)
```

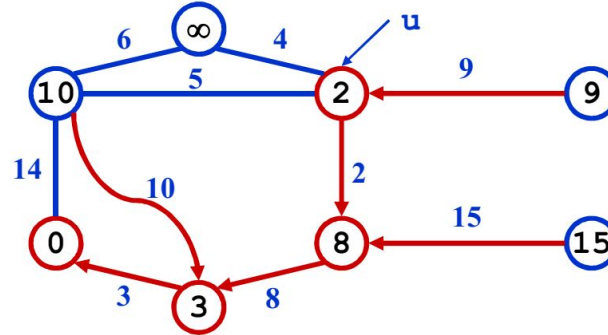
```
    u = ExtractMin(Q);
```

```
    for each v ∈ Adj[u]
```

```
      if (v ∈ Q and w(u,v) < key[v])
```

```
        p[v] = u;
```

```
        key[v] = w(u,v);
```



Prim's Algorithm

```
MST-Prim(G, w, r)
```

```
  Q = V[G];
```

```
  for each  $u \in Q$ 
```

```
     $\text{key}[u] = \infty$ ;
```

```
   $\text{key}[r] = 0$ ;
```

```
   $p[r] = \text{NULL}$ ;
```

```
  while (Q not empty)
```

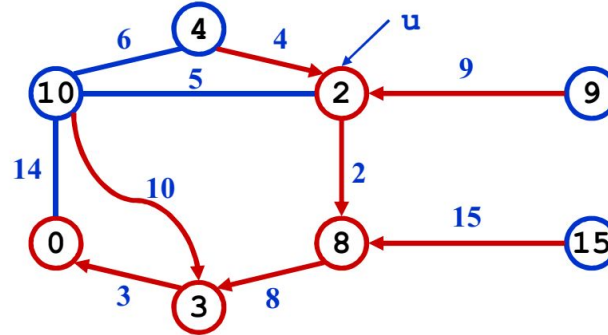
```
     $u = \text{ExtractMin}(Q)$ ;
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u,v) < \text{key}[v]$ )
```

```
         $p[v] = u$ ;
```

```
         $\text{key}[v] = w(u,v)$ ;
```



Prim's Algorithm

```
MST-Prim(G, w, r)
```

```
  Q = V[G];
```

```
  for each  $u \in Q$ 
```

```
     $\text{key}[u] = \infty$ ;
```

```
   $\text{key}[r] = 0$ ;
```

```
   $p[r] = \text{NULL}$ ;
```

```
  while (Q not empty)
```

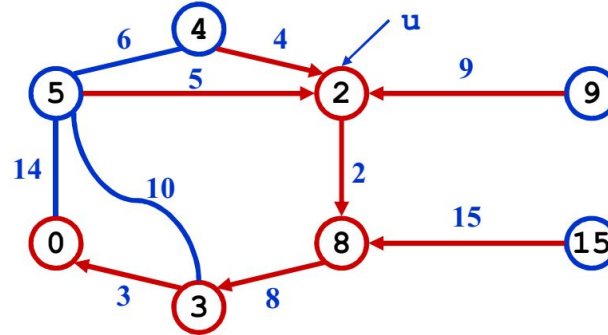
```
     $u = \text{ExtractMin}(Q)$ ;
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u,v) < \text{key}[v]$ )
```

```
         $p[v] = u$ ;
```

```
         $\text{key}[v] = w(u,v)$ ;
```



Prim's Algorithm

```
MST-Prim(G, w, r)
```

```
  Q = V[G];
```

```
  for each  $u \in Q$ 
```

```
     $\text{key}[u] = \infty$ ;
```

```
   $\text{key}[r] = 0$ ;
```

```
   $p[r] = \text{NULL}$ ;
```

```
  while (Q not empty)
```

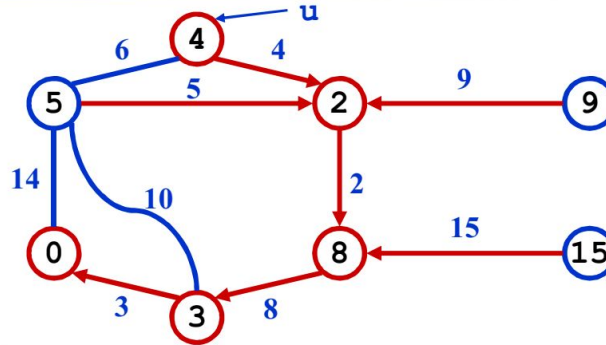
```
     $u = \text{ExtractMin}(Q)$ ;
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )
```

```
         $p[v] = u$ ;
```

```
         $\text{key}[v] = w(u, v)$ ;
```



Prim's Algorithm

```
MST-Prim( $G, w, r$ )
```

```
   $Q = V[G];$ 
```

```
  for each  $u \in Q$ 
```

```
     $\text{key}[u] = \infty;$ 
```

```
   $\text{key}[r] = 0;$ 
```

```
   $p[r] = \text{NULL};$ 
```

```
  while ( $Q$  not empty)
```

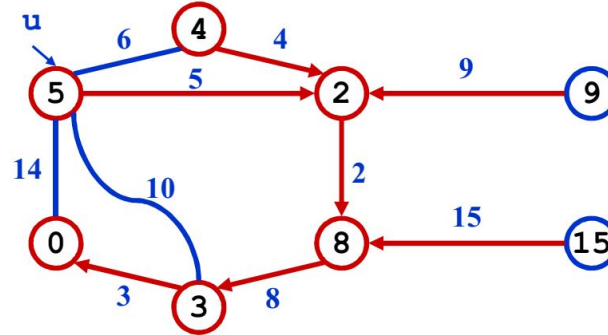
```
     $u = \text{ExtractMin}(Q);$ 
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )
```

```
         $p[v] = u;$ 
```

```
         $\text{key}[v] = w(u, v);$ 
```



Prim's Algorithm

```
MST-Prim(G, w, r)
```

```
  Q = V[G];
```

```
  for each  $u \in Q$ 
```

```
     $\text{key}[u] = \infty$ ;
```

```
   $\text{key}[r] = 0$ ;
```

```
   $p[r] = \text{NULL}$ ;
```

```
  while (Q not empty)
```

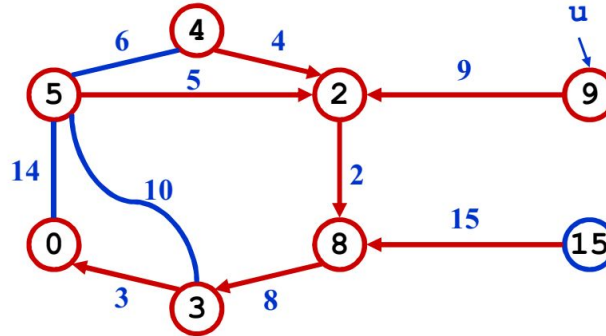
```
     $u = \text{ExtractMin}(Q)$ ;
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u,v) < \text{key}[v]$ )
```

```
         $p[v] = u$ ;
```

```
         $\text{key}[v] = w(u,v)$ ;
```



$$\text{MST-Prim}(G, w, r)$$
$$Q = V[G] ;$$

for each $u \in Q$

```
key[u] = ∞;
```

```
key[r] = 0;
```

```
p[r] = NULL;
```

```
while (Q not empty)
```

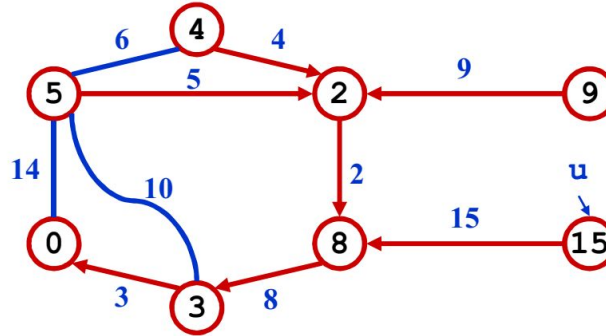
```
u = ExtractMin(Q);
```

for each $v \in \text{Adj}[u]$

```
if (v ∈ Q and w(u, v) < key[v])
```

$p[v] = u;$

```
key[v] = w(u, v);
```



Prim's Algorithm

```
MST-Prim( $G, w, r$ )
```

```
   $Q = V[G];$ 
```

```
  for each  $u \in Q$ 
```

```
     $\text{key}[u] = \infty;$ 
```

```
   $\text{key}[r] = 0;$ 
```

```
   $p[r] = \text{NULL};$ 
```

```
  while ( $Q$  not empty)
```

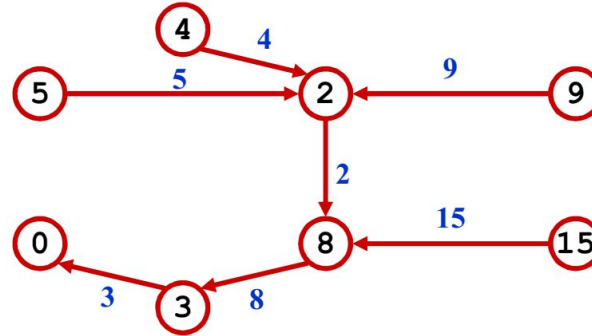
```
     $u = \text{ExtractMin}(Q);$ 
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )
```

```
         $p[v] = u;$ 
```

```
         $\text{key}[v] = w(u, v);$ 
```



MST-Prim(G, w, r)

$Q = V[G];$

 for each $u \in Q$

$key[u] = \infty;$

$key[r] = 0;$

$p[r] = \text{NULL};$

 while (Q not empty)

$u = \text{ExtractMin}(Q);$

 for each $v \in \text{Adj}[u]$

 if ($v \in Q$ and $w(u, v) < key[v]$)

$p[v] = u;$

$key[v] = w(u, v);$

What will be the running time?

Ans: Depends on queue

binary heap: $O(E \lg V)$

Fibonacci heap: $O(V \lg V + E)$

```
import heapq
```

```
def prim(graph):
```

```
    # Select any starting vertex. For simplicity, we choose the first vertex in the graph.
```

```
    start_vertex = list(graph.keys())[0]
```

```
    # Keep track of visited vertices and the edges in the MST.
```

```
    num_vertices = len(graph)
```

```
    visited = [False] * num_vertices
```

```
    mst_edges = []
```

```
    # Initialize the priority queue with edges from the starting vertex.
```

```
    priority_queue = [(weight, start_vertex, neighbor) for neighbor, weight in graph[start_vertex].items()]
```

```
    heapq.heapify(priority_queue)
```

```
    visited[start_vertex] = True
```

```
    while priority_queue:
```

```
        weight, vertex_from, vertex_to = heapq.heappop(priority_queue)
```

```
        if not visited[vertex_to]:
```

```
            visited[vertex_to] = True
```

```
            mst_edges.append((vertex_from, vertex_to, weight))
```

```
            for neighbor, neighbor_weight in graph[vertex_to].items():
```

```
                if not visited[neighbor]:
```

```
                    heapq.heappush(priority_queue, (neighbor_weight, vertex_to, neighbor))
```

```
    return mst_edges
```

Example usage of Prim's algorithm:

```
graph = {  
    'A': {'B': 1, 'C': 4},  
    'B': {'A': 1, 'C': 2, 'D': 5},  
    'C': {'A': 4, 'B': 2, 'D': 1},  
    'D': {'B': 5, 'C': 1}  
}  
  
mst = prim(graph)  
print(mst)  
# Output: [('A', 'B', 1), ('B', 'C', 2), ('C', 'D', 1)]
```

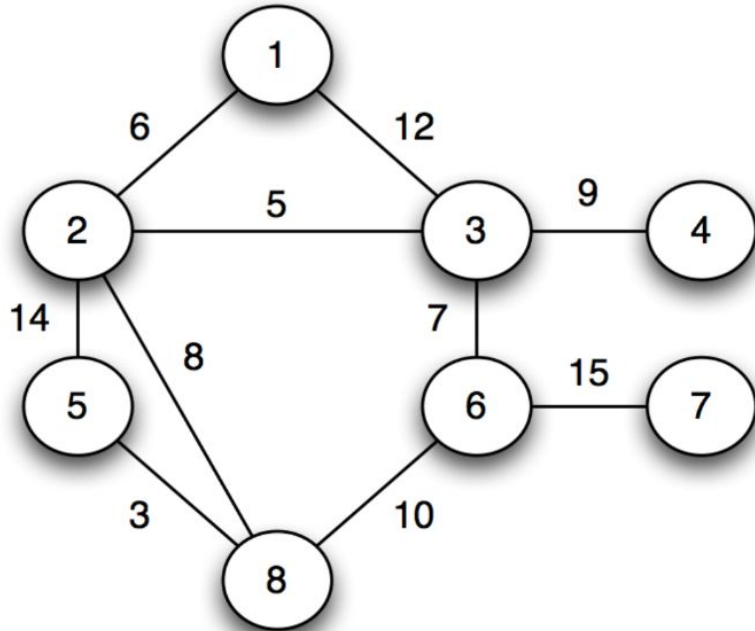
Kruskal's algorithm

MST-KRUSKAL(G, w)

```
1   $A \leftarrow \emptyset$ 
2  for each vertex  $v \in V[G]$ 
3      do MAKE-SET( $v$ )
4  sort the edges of  $E$  into non-decreasing order by weight  $w$ 
5  for each edge  $(u, v) \in E$ , taken in non-decreasing order by weight
6      do if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7          then  $A \leftarrow A \cup \{(u, v)\}$ 
8              UNION( $u, v$ )
9  return  $A$ 
```

Time complexity: $O(E \log E)$

Kruskal's algorithm



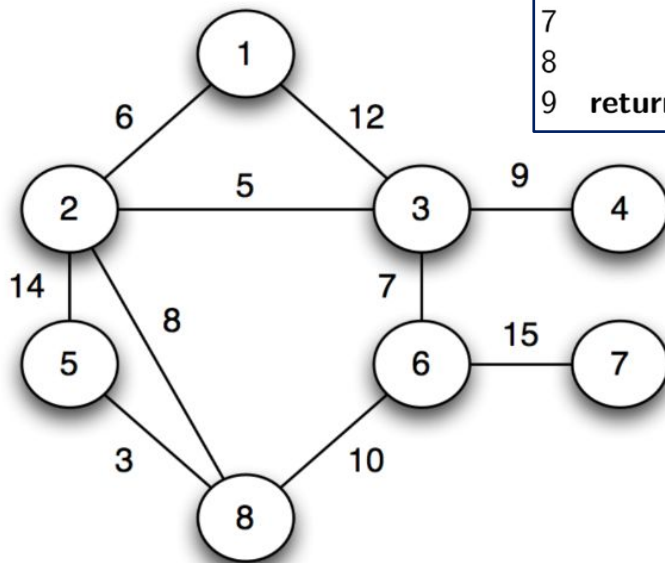
MST-KRUSKAL(G, w)

```
1   $A \leftarrow \emptyset$ 
2  for each vertex  $v \in V[G]$ 
3      do MAKE-SET( $v$ )
4  sort the edges of  $E$  into non-decreasing order by weight  $w$ 
5  for each edge  $(u, v) \in E$ , taken in non-decreasing order by weight
6      do if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7          then  $A \leftarrow A \cup \{(u, v)\}$ 
8              UNION( $u, v$ )
9  return  $A$ 
```

Kruskal's algorithm

$w(u, v)$	(u, v)
3	(5,8)
5	(2,3)
6	(1,2)
7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)

```
MST-KRUSKAL( $G, w$ )  
1   $A \leftarrow \emptyset$   
2  for each vertex  $v \in V[G]$   
3      do MAKE-SET( $v$ )  
4  sort the edges of  $E$  into non-decreasing order by weight  $w$   
5  for each edge  $(u, v) \in E$ , taken in non-decreasing order by weight  
6      do if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )  
7          then  $A \leftarrow A \cup \{(u, v)\}$   
8              UNION( $u, v$ )  
9  return  $A$ 
```



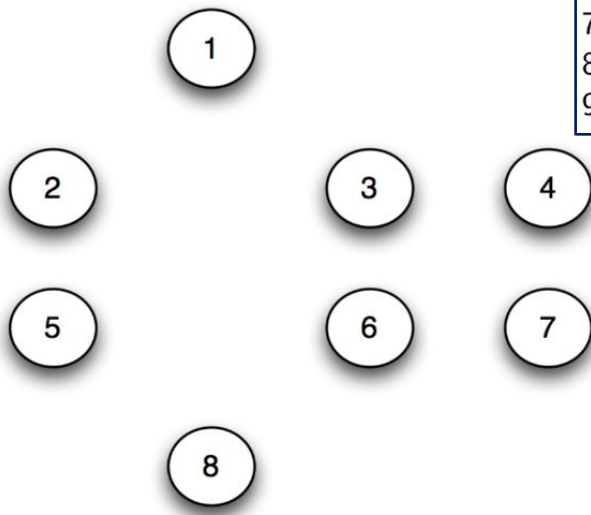
$$|V| = 8$$

$$|E| = 10$$

$$|T| = 0$$

Kruskal's algorithm

$w(u, v)$	(u, v)
3	(5,8)
5	(2,3)
6	(1,2)
7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)



MST-KRUSKAL(G, w)

```
1  $A \leftarrow \emptyset$ 
2 for each vertex  $v \in V[G]$ 
3   do MAKE-SET( $v$ )
4 sort the edges of  $E$  into non-decreasing order by weight  $w$ 
5 for each edge  $(u, v) \in E$ , taken in non-decreasing order by weight
6   do if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7     then  $A \leftarrow A \cup \{(u, v)\}$ 
8         UNION( $u, v$ )
9 return  $A$ 
```

$$|V| = 8$$

$$|E| = 10$$

$$|T| = 0$$

Vertex sets:

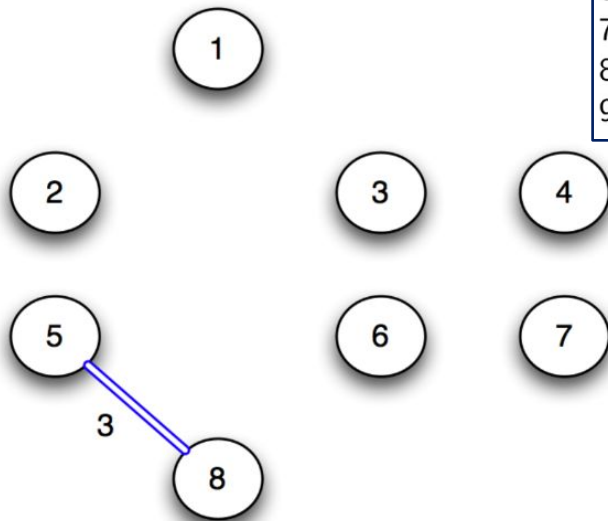
$\{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}, \{7\}, \{8\}$

Kruskal's algorithm

MST-KRUSKAL(G, w)

```
1  $A \leftarrow \emptyset$ 
2 for each vertex  $v \in V[G]$ 
3   do MAKE-SET( $v$ )
4 sort the edges of  $E$  into non-decreasing order by weight  $w$ 
5 for each edge  $(u, v) \in E$ , taken in non-decreasing order by weight
6   do if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7     then  $A \leftarrow A \cup \{(u, v)\}$ 
8         UNION( $u, v$ )
9 return  $A$ 
```

$w(u, v)$	(u, v)
3	(5,8)
5	(2,3)
6	(1,2)
7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)



$$|V| = 8$$

$$|E| = 10$$

$$|T| = 1$$

Vertex sets:

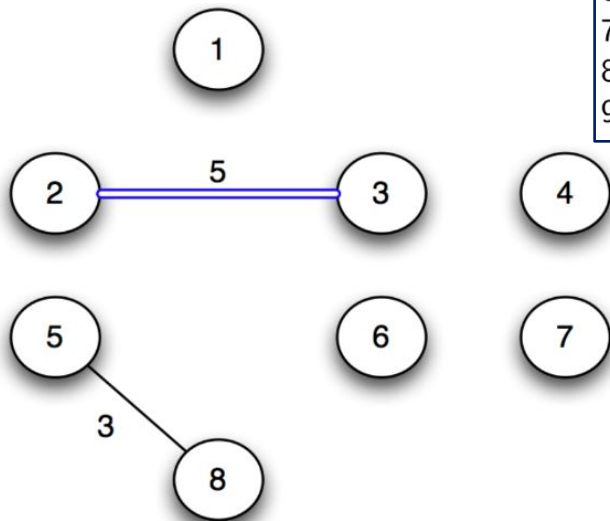
$\{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}, \{7\}, \{8\} \Rightarrow \{1\}, \{2\}, \{3\}, \{4\}, \{5, 8\}, \{6\}, \{7\}$

Kruskal's algorithm

MST-KRUSKAL(G, w)

```
1  $A \leftarrow \emptyset$ 
2 for each vertex  $v \in V[G]$ 
3   do MAKE-SET( $v$ )
4 sort the edges of  $E$  into non-decreasing order by weight  $w$ 
5 for each edge  $(u, v) \in E$ , taken in non-decreasing order by weight
6   do if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7     then  $A \leftarrow A \cup \{(u, v)\}$ 
8         UNION( $u, v$ )
9 return  $A$ 
```

$w(u, v)$	(u, v)
✓ 3	(5,8)
5	(2,3)
6	(1,2)
7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)



$$|V| = 8$$

$$|E| = 10$$

$$|T| = 2$$

Vertex sets:

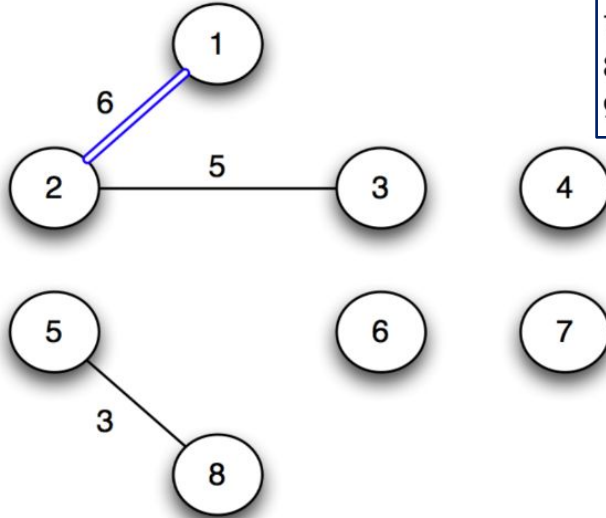
$\{1\}, \{2\}, \{3\}, \{4\}, \{5, 8\}, \{6\}, \{7\} \Rightarrow \{1\}, \{2, 3\}, \{4\}, \{5, 8\}, \{6\}, \{7\}$

Kruskal's algorithm

MST-KRUSKAL(G, w)

```
1  $A \leftarrow \emptyset$ 
2 for each vertex  $v \in V[G]$ 
3   do MAKE-SET( $v$ )
4 sort the edges of  $E$  into non-decreasing order by weight  $w$ 
5 for each edge  $(u, v) \in E$ , taken in non-decreasing order by weight
6   do if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7     then  $A \leftarrow A \cup \{(u, v)\}$ 
8         UNION( $u, v$ )
9 return  $A$ 
```

$w(u, v)$	(u, v)
✓ 3	(5,8)
✓ 5	(2,3)
6	(1,2)
7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)



$|V| = 8$

$|E| = 10$

$|T| = 3$

Vertex sets:

$\{1\}, \{2, 3\}, \{4\}, \{5, 8\}, \{6\}, \{7\} \Rightarrow \{1, 2, 3\}, \{4\}, \{5, 8\}, \{6\}, \{7\}$

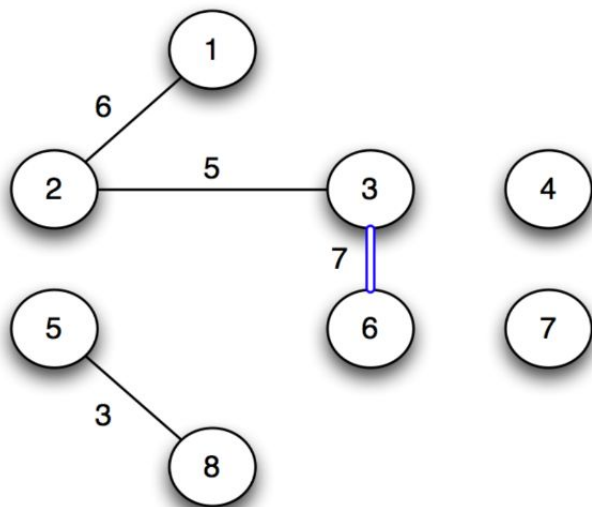
Kruskal's algorithm

MST-KRUSKAL(G, w)

```

1   $A \leftarrow \emptyset$ 
2  for each vertex  $v \in V[G]$ 
3      do MAKE-SET( $v$ )
4  sort the edges of  $E$  into non-decreasing order by weight  $w$ 
5  for each edge  $(u, v) \in E$ , taken in non-decreasing order by weight
6      do if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7          then  $A \leftarrow A \cup \{(u, v)\}$ 
8              UNION( $u, v$ )
9  return  $A$ 
    
```

$w(u, v)$	(u, v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)



$$|V| = 8$$

$$|E| = 10$$

$$|T| = 4$$

Vertex sets:

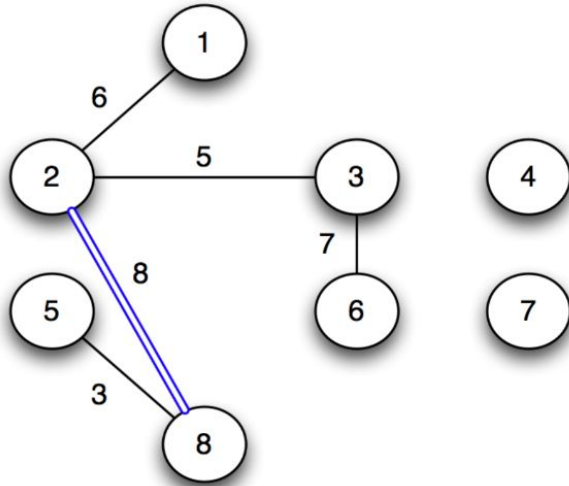
$\{1, 2, 3\}, \{4\}, \{5, 8\}, \{6\}, \{7\} \Rightarrow \{1, 2, 3, 6\}, \{4\}, \{5, 8\}, \{7\}$

Kruskal's algorithm

MST-KRUSKAL(G, w)

```
1  $A \leftarrow \emptyset$ 
2 for each vertex  $v \in V[G]$ 
3   do MAKE-SET( $v$ )
4 sort the edges of  $E$  into non-decreasing order by weight  $w$ 
5 for each edge  $(u, v) \in E$ , taken in non-decreasing order by weight
6   do if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7     then  $A \leftarrow A \cup \{(u, v)\}$ 
8         UNION( $u, v$ )
9 return  $A$ 
```

$w(u, v)$	(u, v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)



$|V| = 8$

$|E| = 10$

$|T| = 5$

Vertex sets:

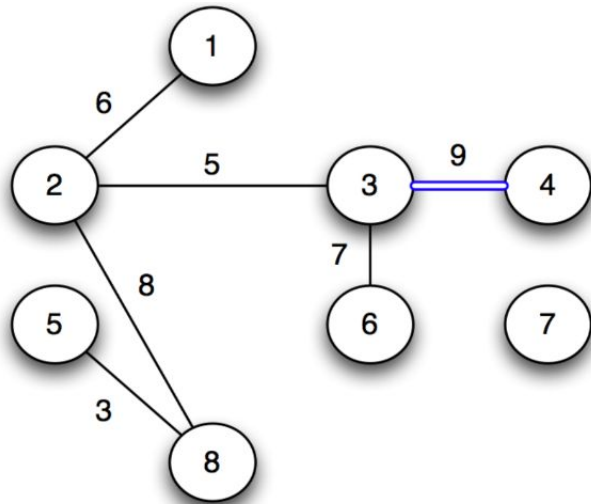
$\{1, 2, 3, 6\}, \{4\}, \{5, 8\}, \{7\} \Rightarrow \{1, 2, 3, 5, 6, 8\}, \{4\}, \{7\}$

Kruskal's algorithm

MST-KRUSKAL(G, w)

```
1  $A \leftarrow \emptyset$ 
2 for each vertex  $v \in V[G]$ 
3   do MAKE-SET( $v$ )
4 sort the edges of  $E$  into non-decreasing order by weight  $w$ 
5 for each edge  $(u, v) \in E$ , taken in non-decreasing order by weight
6   do if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7     then  $A \leftarrow A \cup \{(u, v)\}$ 
8         UNION( $u, v$ )
9 return  $A$ 
```

$w(u, v)$	(u, v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
✓ 8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)



$$|V| = 8$$

$$|E| = 10$$

$$|T| = 6$$

Vertex sets:

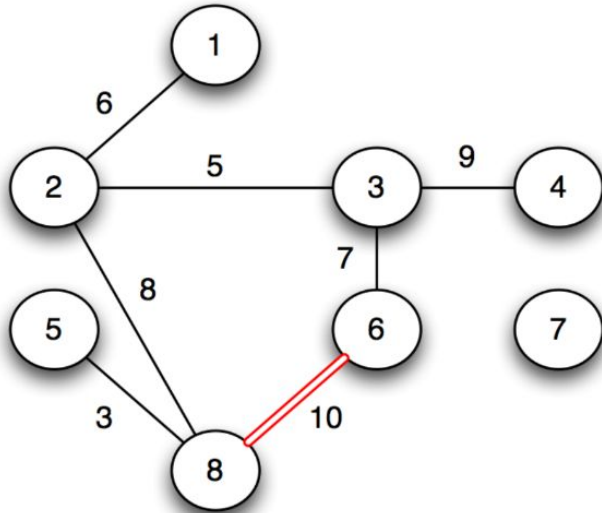
$\{1, 2, 3, 5, 6, 8\}, \{4\}, \{7\} \Rightarrow \{1, 2, 3, 4, 5, 6, 8\}, \{7\}$

Kruskal's algorithm

MST-KRUSKAL(G, w)

```
1   $A \leftarrow \emptyset$ 
2  for each vertex  $v \in V[G]$ 
3      do MAKE-SET( $v$ )
4  sort the edges of  $E$  into non-decreasing order by weight  $w$ 
5  for each edge  $(u, v) \in E$ , taken in non-decreasing order by weight
6      do if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7          then  $A \leftarrow A \cup \{(u, v)\}$ 
8              UNION( $u, v$ )
9  return  $A$ 
```

$w(u, v)$	(u, v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
✓ 8	(2,8)
✓ 9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)



$$|V| = 8$$

$$|E| = 10$$

$$|T| = 6$$

Vertex sets:

$\{1, 2, 3, 4, 5, 6, 8\}, \{7\} \Rightarrow \{1, 2, 3, 4, 5, 6, 8\}, \{7\}$

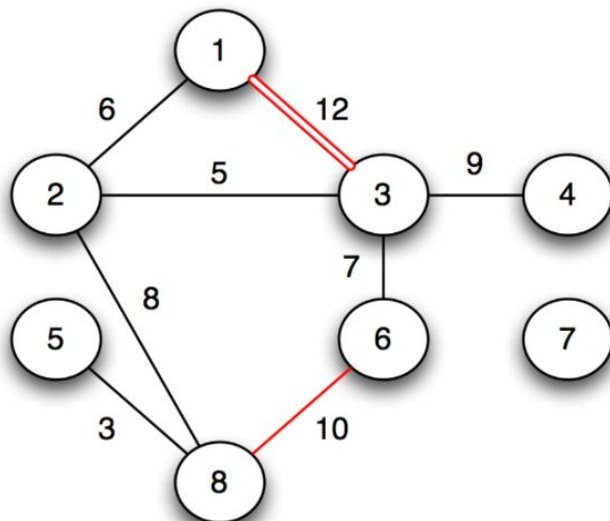
Kruskal's algorithm

MST-KRUSKAL(G, w)

```

1   $A \leftarrow \emptyset$ 
2  for each vertex  $v \in V[G]$ 
3      do MAKE-SET( $v$ )
4  sort the edges of  $E$  into non-decreasing order by weight  $w$ 
5  for each edge  $(u, v) \in E$ , taken in non-decreasing order by weight
6      do if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7          then  $A \leftarrow A \cup \{(u, v)\}$ 
8              UNION( $u, v$ )
9  return  $A$ 
    
```

$w(u, v)$	(u, v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
✓ 8	(2,8)
✓ 9	(3,4)
× 10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)



$$|V| = 8$$

$$|E| = 10$$

$$|T| = 6$$

Vertex sets:

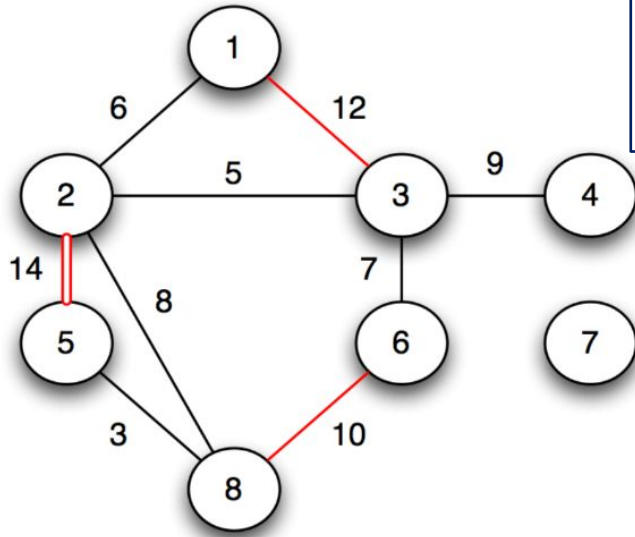
$\{1, 2, 3, 4, 5, 6, 8\}, \{7\} \Rightarrow \{1, 2, 3, 4, 5, 6, 8\}, \{7\}$

Kruskal's algorithm

MST-KRUSKAL(G, w)

```
1   $A \leftarrow \emptyset$ 
2  for each vertex  $v \in V[G]$ 
3      do MAKE-SET( $v$ )
4  sort the edges of  $E$  into non-decreasing order by weight  $w$ 
5  for each edge  $(u, v) \in E$ , taken in non-decreasing order by weight
6      do if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7          then  $A \leftarrow A \cup \{(u, v)\}$ 
8              UNION( $u, v$ )
9  return  $A$ 
```

$w(u, v)$	(u, v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
✓ 8	(2,8)
✓ 9	(3,4)
× 10	(6,8)
× 12	(1,3)
14	(2,5)
15	(6,7)



$|V| = 8$

$|E| = 10$

$|T| = 6$

Vertex sets:

$\{1, 2, 3, 4, 5, 6, 8\}, \{7\} \implies \{1, 2, 3, 4, 5, 6, 8\}, \{7\}$

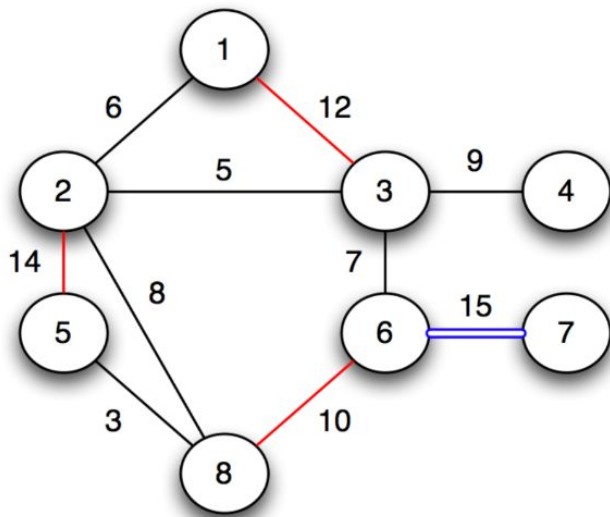
Kruskal's algorithm

MST-KRUSKAL(G, w)

```

1   $A \leftarrow \emptyset$ 
2  for each vertex  $v \in V[G]$ 
3      do MAKE-SET( $v$ )
4  sort the edges of  $E$  into non-decreasing order by weight  $w$ 
5  for each edge  $(u, v) \in E$ , taken in non-decreasing order by weight
6      do if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7          then  $A \leftarrow A \cup \{(u, v)\}$ 
8              UNION( $u, v$ )
9  return  $A$ 
    
```

$w(u, v)$	(u, v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
✓ 8	(2,8)
✓ 9	(3,4)
× 10	(6,8)
× 12	(1,3)
× 14	(2,5)
15	(6,7)



$|V| = 8$

$|E| = 10$

$|T| = 7$

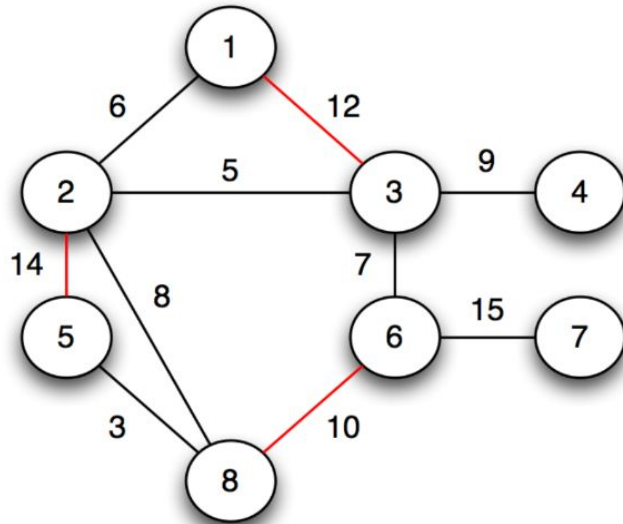
Vertex sets:

$\{1, 2, 3, 4, 5, 6, 8\}, \{7\} \Rightarrow \{1, 2, 3, 4, 5, 6, 7, 8\}$

Kruskal's algorithm

MST-KRUSKAL(G, w)

```
1   $A \leftarrow \emptyset$ 
2  for each vertex  $v \in V[G]$ 
3      do MAKE-SET( $v$ )
4  sort the edges of  $E$  into non-decreasing order by weight  $w$ 
5  for each edge  $(u, v) \in E$ , taken in non-decreasing order by weight
6      do if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7          then  $A \leftarrow A \cup \{(u, v)\}$ 
8              UNION( $u, v$ )
9  return  $A$ 
```



$$|V| = 8$$

$$|E| = 10$$

$$|T| = 7$$

Vertex sets:

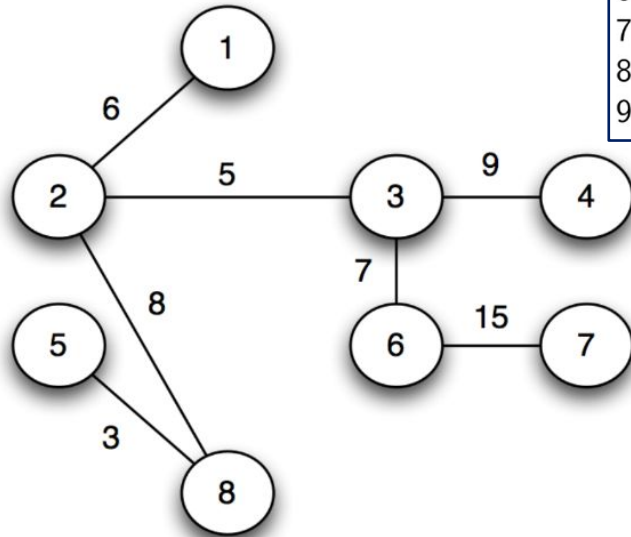
$\{1, 2, 3, 4, 5, 6, 7, 8\}$

Kruskal's algorithm

MST-KRUSKAL(G, w)

```
1   $A \leftarrow \emptyset$ 
2  for each vertex  $v \in V[G]$ 
3      do MAKE-SET( $v$ )
4  sort the edges of  $E$  into non-decreasing order by weight  $w$ 
5  for each edge  $(u, v) \in E$ , taken in non-decreasing order by weight
6      do if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7          then  $A \leftarrow A \cup \{(u, v)\}$ 
8              UNION( $u, v$ )
9  return  $A$ 
```

$w(u, v)$	(u, v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
✓ 8	(2,8)
✓ 9	(3,4)
× 10	(6,8)
× 12	(1,3)
× 14	(2,5)
✓ 15	(6,7)



$$|V| = 8$$

$$|E| = 10$$

$$|T| = 7$$

Vertex sets:

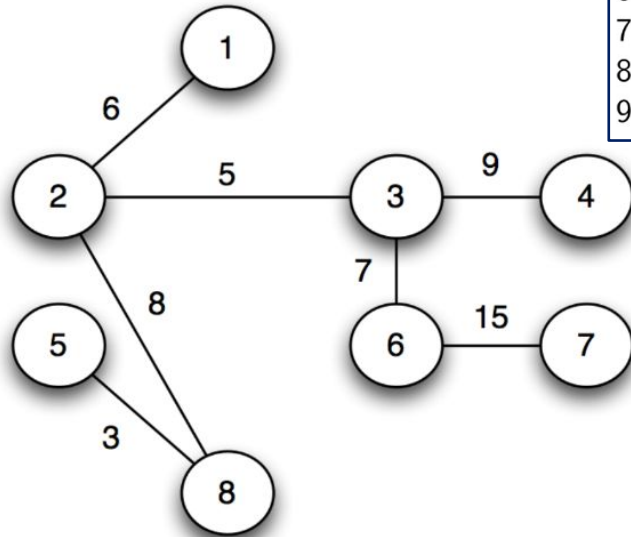
{1, 2, 3, 4, 5, 6, 7, 8}

Kruskal's algorithm

MST-KRUSKAL(G, w)

```
1   $A \leftarrow \emptyset$ 
2  for each vertex  $v \in V[G]$ 
3      do MAKE-SET( $v$ )
4  sort the edges of  $E$  into non-decreasing order by weight  $w$ 
5  for each edge  $(u, v) \in E$ , taken in non-decreasing order by weight
6      do if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7          then  $A \leftarrow A \cup \{(u, v)\}$ 
8              UNION( $u, v$ )
9  return  $A$ 
```

$w(u, v)$	(u, v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
✓ 8	(2,8)
✓ 9	(3,4)
× 10	(6,8)
× 12	(1,3)
× 14	(2,5)
✓ 15	(6,7)



$$|V| = 8$$

$$|E| = 10$$

$$|T| = 7$$

Vertex sets:

{1, 2, 3, 4, 5, 6, 7, 8}

Kruskal's algorithm

MST-KRUSKAL(G, w)

```
1   $A \leftarrow \emptyset$ 
2  for each vertex  $v \in V[G]$ 
3      do MAKE-SET( $v$ )
4  sort the edges of  $E$  into non-decreasing order by weight  $w$ 
5  for each edge  $(u, v) \in E$ , taken in non-decreasing order by weight
6      do if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7          then  $A \leftarrow A \cup \{(u, v)\}$ 
8              UNION( $u, v$ )
9  return  $A$ 
```

